Exercise 3 – Spring

תזכורת: יש באתר הקורס מסמכים הכוללים רשימת בעיות ופתרונות בפרויקטים, מומלץ לדפדף שם לפני שאתם מחפשים ברשת או שולחים מייל!

נביאים: https://classroom.github.com/a/mEXbWGCz

שטראוס: https://classroom.github.com/a/GA17uJkR

Deadline for final submission: 5/7/2026

Goal: build a website combining the Spring technology taught in this course.
המטרה היא להשתמש ב-Spring MVC בלבד (ללא React) ולהעביר את כל ניהול לוגיקת האתר אל הצד שרת (בניגוד ל-React)

The website topic is up to you. We have some suggestions below if you're out of ideas.

You need to produce a fully working website based on the Spring technology taught in this course that fulfill these requirements:

1. A full SpringBoot MVC project : the MVC structure of your project uses Thymeleaf view engine, controllers, beans, dependency injection. The logic is server side, you will write limited Javascript logic if needed (such as validation, polling…)
2. There should be at least 5 major pages. All html pages are built with Thymeleaf (server side dynamically generated pages). You may implement one page in a SPA style (by developing your own REST api) with JS or React **(**)**.
    - For example: landing page, user profile, shopping cart, registration, search, backend/admin…
3. Use sessions : for example to store a cart, game current state, general current state of the user experience
4. Use Beans, dependency injection: make sure to inject beans in your controllers/services (do not access application/session/request scope directly from the http request, do no declare global static objects)
5. Storing persistent data with JPA - mySQL database named "**ex4**" with at least 4 repo beans (SQL tables) with relations.
6. Spring Security that provides user authentication and authorization, and saves you the hassle of implementing user login (a lot of useful material is available on Baeldung's website). Implement user registration unless project scope is very large.

Use optionally: Interceptors/Filters if relevant

Grading criteria

- Scope: if you have a doubt, contact the teacher, scope is crucial, you most likely will include:
    - Login/logout
    - Registration
    - Browsing, advanced searching, multiple browsing states handling
    - Handling multiple DAO with relations
    - Backend (/admin): handling product, users, monitoring, …
    - Depending on the topic: messaging, rating, uploading files etc…
    - For the more adventurous wishing to add a "AI touch": using Spring AI with free models such as Hugging Face (Running Hugging Face models can run

locally via the Spring AI Ollama Integration to completely avoid cloud subscription fees)

- Completeness of the functionality
  - it is better to submit a limited and fully working website, than a pile of code that does not run.
  - It is acceptable to have minor bugs
- Fulfilling technology requirements (should result in very clean and modular code)
- Quality of the UI experience (navigation, design)
- Robustness: server-side integrity (server side validation, thread safe coding, routes/rest api access control), error handling (user feedback, custom error pages), transactions
- your ability to answer any questions about your solution: after submission, some of you may be required to demo their project
- *Please do not submit someone else's code, we hate giving zero grades*

Here are some ideas:

- Any system with user registration, shopping cart, products, orders, product reviews
- A course registration system for students and teachers.
- A system for posting ads, messaging
- A whatsapp kind of chatroom with multiple options, such as creating private chats, handling contact list, sending files

Beware that games might not be the best: you could to get lost in a big client code SPA page, while the goal is to work server side with Spring.

Notes:

- In order to fulfill the use of JPA-SQL, store some application state in the database: user settings and profile, high score, history, messages…
- For any website, you could develop an admin backend with a predefined admin account, to manage anything relevant to the website , for example in the case of a game, you may allow admin to erase the high score, set some game parameters stored in database), consult some history/logfile of actions, handle a store stock etc…
- In order to fulfill the session requirement, you could implement a search page where session is used to remember recent searches and present them nicely in a page for quick access. Another idea is to track the recent actions and provide shortcuts as well. You could also allow the user to "mark" some items for later use.
- Think about adding advanced search in all of your data browsing pages
- If you have a shopping cart, allow shopping without login, then make sure to keep the cart after login
- The project must run on an empty database! If initializations are needed, do it automatically at startup. For example if your system needs an admin account, make sure to create it if it does not exist (see Context listeners for doing that).

Submission:

- **First stage, optional but recommended: on 15/6/2025**: submit **in your repository** a **Word** document with a full description of your project:
  - General explanation
  - A list of main pages and their goal

  - A detailed description of database beans: describe each classe and its relations (you can present code in your repo)

  This document is not a contract; you will be able to make any modification later on. If present, you will receive feedback from the teacher by the 20/6, your repository README.md file will be modified so make sure to download it after this date.

- **On 29/6/2026 at 13:00: demo at the lab, schedule will be published**
  Demo/Video:
  If your project is ready to be demo-ed on that day at the college. You'll get my feedback immediately and will have few more days to fix things if needed **until July 5th.**

In addition to your code, you should submit a short recording demo of your website (or link to), nothing professional, it should be very informal:

- Length of the recording limited to 12 minutes
- Continuous straight recording, no editing/cut
- All participants must present, if you are a team of 2, make sure to split the task of presenting, show yourself in the video (Zoom can be used easily for this).
- Upload the recording to your repo or put a link in your README.md

Submit also a SQL dump of your database (export your database under phpMyAdmin or other tool) with enough data for us to play with the website.

Your README.md should explain:

  - The general functionality of the website
  - Compile and run instructions **(*)** (so we can run them by clicking on the readme file)
  - Anything we need to know
    - Any credentials (for example the admin login and password)
    - Credentials of the SQL dump if it includes users/passwords

(*) We require to compile and run your project with:

- **mvnw clean package**
- **mvnw spring-boot:run**

(**) For those wishing to include a React page under Spring auth system, check out https://github.com/solangek/react-with-spring-login

List of course repos worth visiting:
https://github.com/solangek?tab=repositories&q=spring

Good luck!